

WORLDPAY INTERVIEW GUIDE — PART 2

Sanket Ajay Chopade

Resume Deep-Dive · Troubleshooting · Docker · Linux · Scripting

DevSecOps · Monitoring · AWS · Rapid-Fire · Smart Questions

100+ Questions | Full Structured Answers | All Commands in Correct Syntax



Table of Contents

1. Resume Deep-Dive — CI/CD & Jenkins
2. Resume Deep-Dive — Kubernetes
3. Resume Deep-Dive — Terraform & IaC
4. Resume Deep-Dive — AWS Core
5. Scenario-Based Troubleshooting (CI/CD, K8s, AWS)
6. Docker & Containers
7. Linux Administration
8. Scripting — Shell & Python
9. DevSecOps
10. Monitoring & Observability
11. Cross / Trap Questions
12. Rapid-Fire Round
13. Smart Questions YOU Should Ask

1. Resume Deep-Dive — CI/CD & Jenkins

Q: You reduced deployment time by 77% — exactly how did you achieve this?

■ **Project Walkthrough — Problem → Tech Stack → Role → Challenges → Solution → Result**

■ **Before State:**

Deployment took 45 minutes. Stages ran sequentially. No caching. Manual approval gates at every stage. Docker images rebuilt from scratch every time.

■ **Analysis:**

I profiled each pipeline stage using Jenkins timestamps plugin and found 60% of time was wasted in sequential test execution and redundant Docker layer pulls.

■ **Optimization 1 — Parallel Stages:**

Split unit tests, integration tests, and static analysis into parallel Jenkins stages running simultaneously.

```
// Jenkinsfile — Parallel test execution stage('Test & Scan') { parallel { stage('Unit Tests') { steps { sh 'mvn test -Dgroups=unit' } } stage('SonarQube Analysis') { steps { withSonarQubeEnv('sonar-server') { sh 'mvn sonar:sonar' } } } stage('Trivy Scan') { steps { sh 'trivy image --exit-code 1 --severity HIGH,CRITICAL ${IMAGE}' } } } }
```

■ **Optimization 2 — Docker Layer Caching:**

Used --cache-from flag to reuse previously built image layers. Multi-stage Dockerfile separates dependency download from app build — dependency layer cached between builds.

```
# Pull previous image as cache before building docker pull
${ECR_REGISTRY}/${SERVICE}:latest || true docker build \ --cache-from
${ECR_REGISTRY}/${SERVICE}:latest \ -t ${ECR_REGISTRY}/${SERVICE}:${GIT_COMMIT} .
```

■ **Optimization 3 — Maven Dependency Cache:**

Mounted Maven .m2 cache as a persistent volume on Jenkins agent — eliminated re-downloading 200+ dependencies on every build.

■ **Optimization 4 — Automated Rollouts:**

Replaced manual kubectl apply with Helm upgrade --wait, removing human handoff delay between build and deploy stages.

■ **Result:**

Pipeline time: 45 minutes → 10 minutes (77% reduction). Release frequency: 1x/week → 3x/week. Deployment success rate: 89% → 98%.

Q: Walk me through your Jenkins pipeline end-to-end.

■ **Project Walkthrough**

■ **Stage 1 — Checkout:**

GitHub webhook triggers Jenkins on PR merge to main. Jenkinsfile is read from repository root.

■ **Stage 2 — Build:**

Maven compiles Java source, runs unit tests, produces JAR artifact.

■ **Stage 3 — Docker Build:**

Multi-stage Dockerfile builds optimized image. Image tagged with git commit SHA for immutable traceability.

■ Stage 4 — Security Scan:

Trivy scans image for CVEs. SonarQube performs static analysis. Pipeline fails if HIGH/CRITICAL CVEs found or quality gate fails.

■ Stage 5 — Push to ECR:

Authenticated push using IAM role on Jenkins agent (no hardcoded credentials).

■ Stage 6 — Deploy to EKS:

Helm upgrade --install deploys to production namespace. --wait flag makes Jenkins wait for rollout completion before marking success.

■ Stage 7 — Smoke Test:

Automated curl health check hits /actuator/health endpoint. Alert fires in Grafana if error rate spikes above 1% within 5 minutes of deployment.

■ Stage 8 — Notify:

Slack notification with deployment status, image tag, and Grafana dashboard link sent to #deployments channel.

Q: How do you handle failures in a CI/CD pipeline?

■ Root Cause Method — Identify → Diagnose → Fix → Prevent

■ Stage Failure Detection:

Each Jenkins stage has a post { failure { } } block that captures logs, sends Slack alerts, and marks the build as failed with a descriptive message.

■ Test Failures:

Build stops immediately. Test reports archived to Jenkins. Developer notified with link to failed test output. No deployment proceeds.

■ Deployment Failure:

If Helm upgrade fails (pod not ready within timeout), the pipeline runs helm rollback automatically.

```
// Auto-rollback on deployment failure stage('Deploy') { steps { script { def status = sh( script: 'helm upgrade --install myapp ./helm --wait --timeout 5m0s', returnStatus: true ) if (status != 0) { sh 'helm rollback myapp 0' // 0 = previous revision error('Deployment failed - rolled back to previous version') } } } }
```

■ Post-Failure Actions:

Jenkins archives build artifacts, test reports, and Docker build logs. CloudWatch alarm triggers if deployment frequency drops (sign of repeated failures).

Q: How do you ensure rollback in your pipeline?

■ Concept Explanation — Definition → How it Works → Example

■ Layer 1 — Helm Rollback:

Every Helm release stores revision history. kubectl rollout undo and helm rollback both work in under 60 seconds.

■ Layer 2 — Image Tag Immutability:

Every image is tagged with git commit SHA. Rolling back means redeploying the previous SHA tag — no ambiguity about what version you are running.

■ Layer 3 — Kubernetes Rolling Update:

maxUnavailable: 0 in the deployment strategy means old pods stay up until new pods are fully healthy. If new pods fail readiness probes, the rollout stops and old pods continue serving traffic.

```
# Manual rollback commands # Option 1: Helm rollback to previous release helm rollback myapp 0 -n production # Option 2: Kubernetes rollout undo kubectl rollout undo deployment/payment-service -n production # Verify rollback completed kubectl rollout status deployment/payment-service -n production # Check which image is now running kubectl get deployment payment-service -n production \ -o jsonpath='{.spec.template.spec.containers[0].image}'
```

2. Resume Deep-Dive — Kubernetes

Q: Explain your EKS architecture in the ApnaKart project.

■ **Top-Down Design — Requirements → Architecture → Components → Scaling → Trade-offs**

■ **VPC Design:**

Custom VPC with 3 public subnets (ALB, NAT Gateway) and 3 private subnets (EKS nodes, RDS) across 3 AZs for high availability. No EKS worker nodes in public subnets.

■ **EKS Control Plane:**

Managed EKS control plane (AWS-managed). Worker nodes are EC2 instances (t3.medium) in private subnets, part of a managed node group with Cluster Autoscaler for automatic scaling.

■ **Ingress:**

AWS Application Load Balancer (ALB) in public subnets, terminating HTTPS (ACM certificate). AWS Load Balancer Controller translates Kubernetes Ingress objects to ALB listener rules.

■ **Service Mesh:**

ClusterIP services for inter-service communication. No service mesh — Kubernetes NetworkPolicies enforce traffic isolation between namespaces.

■ **Data Layer:**

RDS PostgreSQL in private subnets. Each microservice has its own RDS instance (database-per-service). Kubernetes Secrets store DB credentials (backed by AWS Secrets Manager in production).

■ **Observability:**

Prometheus (in-cluster) scrapes metrics via ServiceMonitor. Grafana dashboards in cluster. Datadog agent as DaemonSet on all nodes. CloudWatch Container Insights enabled.

```
# Verify EKS cluster and node health kubectl get nodes -o wide kubectl get pods -A | grep -v Running # Check Cluster Autoscaler status kubectl logs -n kube-system \ -l app=cluster-autoscaler --tail=20 # Check ALB ingress kubectl get ingress -n production
```

Q: How did you achieve zero-downtime deployments?

■ **Concept Explanation + ■ Root Cause Method**

■ **Mechanism 1 — Rolling Update Strategy:**

maxSurge: 1 allows one extra pod. maxUnavailable: 0 ensures no pod is removed until replacement is fully ready.

■ **Mechanism 2 — Readiness Probes:**

Kubernetes only sends traffic to a new pod after its readiness probe returns HTTP 200. This prevents traffic hitting a pod before the app has fully started.

■ **Mechanism 3 — Graceful Shutdown:**

terminationGracePeriodSeconds: 30 gives in-flight requests 30 seconds to complete before the pod is terminated.

■ **Mechanism 4 — Pod Disruption Budget:**

PDB ensures at least 2 replicas of critical services are always available during node maintenance or rolling updates.

```
# deployment.yaml – Zero-downtime configuration apiVersion: apps/v1 kind: Deployment
spec: replicas: 3 strategy: type: RollingUpdate rollingUpdate: maxSurge: 1
maxUnavailable: 0 template: spec: terminationGracePeriodSeconds: 30 containers: - name:
payment-service readinessProbe: httpGet: path: /actuator/health/readiness port: 8080
initialDelaySeconds: 20 periodSeconds: 5 failureThreshold: 3 livenessProbe: httpGet:
path: /actuator/health/liveness port: 8080 initialDelaySeconds: 30 periodSeconds: 10 ---
# pdb.yaml – Pod Disruption Budget apiVersion: policy/v1 kind: PodDisruptionBudget
metadata: name: payment-service-pdb spec: minAvailable: 2 selector: matchLabels: app:
payment-service
```

Q: What happens when a pod crashes? Walk me through the full lifecycle.

■ Concept Explanation — Definition → How it Works → Example

■ Step 1 — Crash Detected:

Container exits with non-zero exit code. kubelet on the node detects the crash immediately.

■ Step 2 — Restart Policy:

With restartPolicy: Always (default), kubelet restarts the container. First restart: immediate. Second restart: 10s delay. Third: 20s. Exponential backoff up to 5 minutes — this is CrashLoopBackOff.

■ Step 3 — Liveness Probe:

If the container is running but unhealthy (liveness probe fails 3 times), kubelet kills the container and restarts it — same as a crash.

■ Step 4 — Readiness Probe:

While restarting, readiness probe fails. Kubernetes removes the pod from the Service endpoints — no traffic is routed to the crashing pod. Other healthy replicas absorb the load.

■ Step 5 — Self-Healing:

If the restart resolves the issue (e.g., transient DB connection failure), the pod recovers automatically. If it cannot recover, it stays in CrashLoopBackOff until the root cause is fixed.

■ Step 6 — Events & Alerting:

Kubernetes Events record the crash. Prometheus kube-state-metrics exposes kube_pod_container_status_restarts_total. Alertmanager fires alert if restart count exceeds threshold.

```
# Monitor pod restarts in real time kubectl get pods -n production -w # Check restart
count kubectl get pod -n production \ -o
jsonpath='{.status.containerStatuses[0].restartCount}' # Get crash logs from previous
container instance kubectl logs -n production --previous
```

3. Resume Deep-Dive — Terraform & IaC

Q: How is your Terraform code structured — modules, state, etc.?

■ **Concept Explanation — Definition → How it Works → Example**

■ Directory Structure:

```
/terraform
```

/modules/vpc/ — reusable VPC module

/modules/eks/ — reusable EKS module

/modules/rds/ — reusable RDS module

/envs/staging/ — staging environment

/envs/production/ — production environment

Each env has: main.tf, variables.tf, outputs.tf, backend.tf

■ Module Design:

Each module accepts input variables and exposes output values. Environments compose modules — production just passes different variable values (larger instance types, more replicas) to the same module code.

■ State Management:

Remote state stored in S3 with DynamoDB locking. Each environment has its own state file key to prevent cross-environment blast radius.

■ Variable Hygiene:

Sensitive values (DB passwords, API keys) passed via environment variables or AWS Secrets Manager — never hardcoded in .tf files. .tfvars files excluded from Git via .gitignore.

```
# envs/production/backend.tf terraform { backend "s3" { bucket = "myapp-terraform-state"
key = "production/eks/terraform.tfstate" region = "ap-south-1" dynamodb_table =
"terraform-lock" encrypt = true } } # envs/production/main.tf module "eks" { source =
"../../modules/eks" cluster_name = var.cluster_name node_group_size =
var.node_group_size vpc_id = module.vpc.vpc_id subnet_ids =
module.vpc.private_subnet_ids }
```

Q: How do you handle secrets in Terraform?

■ **Compare & Justify — Problem → Options → Chosen Solution → Trade-offs**

■ Problem:

Terraform configurations often need sensitive values (DB passwords, API keys) to provision resources. Storing these in .tf files or .tfvars in Git is a security risk.

■ Option 1 — Environment Variables:

Set TF_VAR_db_password=secret before running apply. Terraform reads it automatically. Safe but requires manual setup per environment.

■ Option 2 — AWS Secrets Manager + Data Source:

Store secret in AWS Secrets Manager. Terraform reads it at apply time using a data source. Preferred for team environments.


```
# Fetch secret from AWS Secrets Manager in Terraform data
"aws_secretsmanager_secret_version" "db_pass" { secret_id = "prod/myapp/db-password" }
resource "aws_db_instance" "main" { password =
data.aws_secretsmanager_secret_version.db_pass.secret_string # ... other config } # The
secret value is NEVER written to the .tf file or committed to Git
```

■ Option 3 — Mark as Sensitive:

Use sensitive = true in variable declaration. Terraform redacts the value from plan output and logs.

■ Chosen Approach:

AWS Secrets Manager for all production secrets. Environment variables for local development. sensitive = true on all secret variables as an additional safety layer.

Q: How do you manage Terraform state remotely in a team?

■ Concept Explanation

■ S3 for Storage:

State file stored in versioned S3 bucket with AES-256 encryption. Versioning allows recovering from accidental state corruption.

■ DynamoDB for Locking:

Before any plan or apply, Terraform writes a lock record to DynamoDB. If another engineer is running apply simultaneously, the second one gets: Error: state is locked by another process. Prevents race conditions.

■ Workspace Isolation:

Each environment (dev/staging/prod) has a separate state file key. A mistake in dev cannot corrupt prod state.

■ CI/CD Integration:

CI pipeline runs terraform plan and posts output as a PR comment. Terraform apply only runs after manual approval in the pipeline. This ensures no one applies changes directly from their laptop without review.

```
# Setup commands for team state terraform init # Always plan before apply terraform plan
-out=tfplan # Apply only the reviewed plan terraform apply tfplan # If someone's apply
crashed and left a lock: terraform force-unlock # List all state resources terraform
state list # Move a resource between states (e.g., during refactor) terraform state mv \
module.old.aws_s3_bucket.data \ module.new.aws_s3_bucket.data
```

4. Resume Deep-Dive — AWS Core

Q: Explain your VPC architecture.

■ ■ Top-Down Design

■ CIDR Block:

VPC: 10.0.0.0/16. Provides 65,536 IP addresses. Split into public and private subnets across 3 AZs for high availability.

■ Public Subnets (10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24):

Internet-facing resources: Application Load Balancer, NAT Gateways. Internet Gateway attached to VPC. Route table: 0.0.0.0/0 -> Internet Gateway.

■ Private Subnets (10.0.10.0/24, 10.0.11.0/24, 10.0.12.0/24):

EKS worker nodes, RDS instances. Route table: 0.0.0.0/0 -> NAT Gateway. Cannot be reached directly from the internet.

■ NAT Gateway:

One NAT Gateway per AZ (for high availability). Allows private subnet resources to initiate outbound internet connections (e.g., pull Docker images from ECR) without being reachable inbound.

■ Security Groups:

Stateful firewall at instance level. ALB SG: allow 443 inbound from internet. EKS nodes SG: allow traffic from ALB SG only. RDS SG: allow port 5432 from EKS nodes SG only.

Q: Difference between ALB and NLB?

■ ■ Compare & Justify

■ ALB — Application Load Balancer (Layer 7):

Operates at HTTP/HTTPS level

Routes based on URL path, host header, query string

Supports WebSockets, HTTP/2, gRPC

Integrates with AWS WAF for web application firewall

Use for: web apps, microservices, REST APIs, Kubernetes Ingress

■ NLB — Network Load Balancer (Layer 4):

Operates at TCP/UDP/TLS level

Handles millions of requests per second with ultra-low latency (~100ms)

Preserves client source IP natively

Supports static IP / Elastic IP per AZ

Use for: gaming, IoT, financial trading systems requiring ultra-low latency

■ My Choice:

ALB for all ApnaKart microservices — path-based routing (/api/payment → payment-service, /api/orders → order-service) and WAF integration for security. NLB would be used if we had a real-time payment processing engine requiring sub-millisecond latency.

Q: How does IAM role-based access work in AWS?

■ Concept Explanation — Definition → How it Works → Example

■ Definition:

IAM Roles are AWS identities with attached permission policies. Unlike users, roles are assumed temporarily and issue short-lived credentials. No long-term access keys to leak.

■ How It Works:

EC2 instance or EKS pod assumes a role via Instance Metadata Service (IMDS) or IRSA (IAM Roles for Service Accounts). AWS STS issues temporary credentials (15-minute expiry, auto-renewed).

■ IRSA in EKS:

Each Kubernetes service account is annotated with an IAM role ARN. Only pods using that service account can assume that role. Follows least privilege — the Prometheus pod only gets CloudWatch read permissions, not full admin.

```
# Annotate Kubernetes service account with IAM role (IRSA) kubectl annotate
serviceaccount prometheus-sa \ -n monitoring \
eks.amazonaws.com/role-arn=arn:aws:iam::123456789012:role/PrometheusCloudWatchRole # IAM
Trust Policy allows EKS OIDC to assume the role { "Version": "2012-10-17", "Statement":
[ { "Effect": "Allow", "Principal": { "Federated":
"arn:aws:iam::123456789012:oidc-provider/..." }, "Action":
"sts:AssumeRoleWithWebIdentity", "Condition": { "StringEquals": { "...:sub":
"system:serviceaccount:monitoring:prometheus-sa" } } ] }
```

Q: How do services inside private subnets access the internet?

■ Concept Explanation

■ NAT Gateway:

Private subnet route table has a rule: 0.0.0.0/0 -> NAT Gateway ID. NAT Gateway is in the public subnet. It translates the private IP to its own public Elastic IP, sends the request to the internet, and returns the response.

■ Use Cases:

EKS worker nodes pulling Docker images from ECR (or DockerHub). Application pods calling external payment APIs. Terraform downloading provider plugins during apply.

■ Cost Consideration:

NAT Gateway charges per hour + per GB of data processed. For high-volume data transfer, VPC endpoints (for AWS services like S3, ECR, DynamoDB) are much cheaper and don't need the internet at all.

```
# Create VPC endpoint for ECR (avoids NAT Gateway cost for image pulls) aws ec2
create-vpc-endpoint \ --vpc-id vpc-xxxxxxx \ --service-name
com.amazonaws.ap-south-1.ecr.dkr \ --vpc-endpoint-type Interface \ --subnet-ids
subnet-xxx subnet-yyy \ --security-group-ids sg-xxxxxxx
```

Q: Explain Route53 routing policies.

■ Concept Explanation

■ Simple:

One record, one IP. No health checks. Used for: single-server non-critical resources.

■ Weighted:

Route X% of traffic to one target, Y% to another. Used for: gradual traffic shifting during canary deployments (route 5% to new version).

■ **Latency-Based:**

Routes users to the AWS region with lowest latency to them. Used for: multi-region apps where you want users auto-routed to the closest region.

■ **Failover:**

Primary and secondary records. Route53 health checks the primary. If it fails, traffic automatically routes to secondary. Used for: active-passive DR setup.

■ **Geolocation:**

Routes based on user's geographic location. Used for: compliance (EU users must stay in eu-west-1) or localized content.

■ **My Usage:**

Failover policy for DR — primary record points to ap-south-1 ALB, secondary to ap-southeast-1 DR ALB. Route53 health check monitors the primary ALB every 30 seconds.

5. Scenario-Based Troubleshooting

Q: Pipeline is failing intermittently — how do you debug?

■ **Action Plan Method** — Clarify → Immediate Action → Investigation → Communication → Prevention

■ Clarify:

Is it failing at the same stage every time, or different stages? Is it time-correlated (always at peak hours)? When did intermittent failures start?

■ Common Causes of Intermittent Failures:

Flaky tests (timing-dependent unit tests)

Jenkins agent resource exhaustion (OOM, disk full)

Network timeouts (Docker registry pulls, Maven repository)

Race conditions in parallel stage execution

External API rate limits (SonarQube, Trivy DB updates)

```
# Check Jenkins agent disk space df -h /var/jenkins_home # Check agent memory usage
during build free -h # Check build logs for timeout patterns grep -i 'timeout\|connection
refused\|FAIL' build.log # Re-run build with verbose Maven output mvn clean test -X 2>&1
| tail -100
```

■ Fix:

For flaky tests: add retries or fix test logic. For disk: add docker system prune as post-build step. For network: increase timeout in Jenkinsfile and add retry blocks.

```
// Add retry logic for network-sensitive steps stage('Push to ECR') { steps { retry(3) {
sh 'docker push ${IMAGE}' } } }
```

Q: Deployment succeeded but application is not working — what will you check?

■ Root Cause Method

■ Check 1 — Pod Status:

Deployment shows ready but is the pod actually serving traffic?

```
kubectl get pods -n production -l app=my-service kubectl describe pod -n production #
Look for: readiness probe failures, OOMKilled, wrong image
```

■ Check 2 — Service & Endpoints:

Is the Kubernetes Service pointing to the correct pod IPs?

```
kubectl get endpoints my-service -n production # If endpoints list is empty, readiness
probe is failing # Pods are running but not passing readiness check
```

■ Check 3 — Ingress & ALB:

Is the ALB routing to the correct target group?

```
kubectl describe ingress my-ingress -n production aws elbv2 describe-target-health \
--target-group-arn
```

■ Check 4 — Application Logs:

Even if the pod is running, the application inside might have an error.

```
kubectl logs -n production --tail=50 # Look for: DB connection error, missing env var, startup exception
```

■ Check 5 — ConfigMap/Secret:

New deployment might reference a ConfigMap or Secret key that does not exist or has wrong value.

■ Check 6 — New Image:

If the image tag changed, verify the new image is not accidentally running old code (check image digest, not just tag).

Q: EC2 instance is unreachable — what will you check?

■ Root Cause Method — Identify → Diagnose → Fix → Prevent

■ Step 1 — AWS Console Checks:

EC2 Status Checks: System reachability check (AWS hardware), Instance status check (OS-level). If system check fails, it is a hardware issue — stop/start instance (migrates to new host).

■ Step 2 — Security Groups:

Verify inbound rules allow your IP on port 22 (SSH). SGs are stateful — if inbound is blocked, connection is refused/timed out.

■ Step 3 — Network ACL:

NACLs are stateless. Both inbound AND outbound rules must allow traffic. Unlike SGs, NACLs can block based on IP range.

■ Step 4 — Route Table:

Instance in private subnet? Verify NAT Gateway is in route table. Is the Internet Gateway attached for public subnet instances?

■ Step 5 — Key Pair:

SSH key mismatch causes Permission denied (publickey). Use EC2 Instance Connect or SSM Session Manager as alternatives.

■ Step 6 — Instance State:

Is the instance actually running? Check CloudWatch metrics — is CPU at 100% (overloaded)?

```
# Connect via SSM (no SSH key needed, works even if SG blocks port 22) aws ssm
start-session --target i-xxxxxxxxxxxxxxxx # Check instance console output for boot
errors aws ec2 get-console-output --instance-id i-xxxxxxx # Verify SG rules aws ec2
describe-security-groups --group-ids sg-xxxxxxx # Ping/traceroute from another instance
in same VPC ping 10.0.10.50 traceroute 10.0.10.50
```

Q: Sudden spike in AWS cost — how do you analyze it?

■ Action Plan Method

■ Step 1 — AWS Cost Explorer:

Filter by service and time period. Identify which service spiked (EC2, NAT Gateway, data transfer, RDS).

■ Step 2 — Common Culprits:

NAT Gateway: Unoptimized data transfer — use VPC endpoints for S3/ECR

EC2: Auto Scaling group scaled out and did not scale back — check ASG min/max

RDS: Multi-AZ enabled unintentionally or IOPS provisioned too high

Data Transfer: Cross-AZ or cross-region traffic — use same-AZ resources

S3: GetObject requests spiked — check for crawler or misconfigured CDN

■ Step 3 — Set Budgets & Alerts:

AWS Budgets with 80% and 100% thresholds. Anomaly Detection in Cost Explorer for automatic spike detection.

```
# Enable Cost Explorer programmatically aws ce get-cost-and-usage \ --time-period  
Start=2026-04-01,End=2026-04-21 \ --granularity DAILY \ --metrics BlendedCost \  
--group-by Type=DIMENSION,Key=SERVICE
```

6. Docker & Containers

Q: What are multi-stage builds? Why did you use them?

■ Concept Explanation — Definition → How it Works → Example

■ Definition:

Multi-stage builds use multiple FROM statements in one Dockerfile. Each stage can use a different base image. Only the final stage is included in the output image — build tools are discarded.

■ Why:

Without multi-stage builds, a Java app image would include the full JDK (400MB+), Maven, source code, and test files. With multi-stage: the final image only contains the JRE and the compiled JAR.

■ Result from Flight Reservation:

Image size reduced by 60% (e.g., from 650MB to 260MB). Smaller images: faster ECR push/pull, smaller attack surface, less disk usage on EKS nodes.

```
# Multi-stage Dockerfile for Spring Boot application # Stage 1: Build (uses full JDK +
Maven) FROM maven:3.9-eclipse-temurin-17 AS builder WORKDIR /app COPY pom.xml . RUN mvn
dependency:go-offline # Cache deps layer separately COPY src ./src RUN mvn clean package
-DskipTests # Stage 2: Runtime (uses slim JRE only – NO Maven, NO source code) FROM
eclipse-temurin:17-jre-alpine WORKDIR /app COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Q: Difference between CMD and ENTRYPOINT in Docker?

■ Concept Explanation — Definition → How it Works → Example

■ ENTRYPOINT:

Defines the executable that always runs when the container starts. Cannot be overridden by docker run arguments — only by --entrypoint flag.

■ CMD:

Provides default arguments to the ENTRYPOINT. Can be completely overridden by arguments passed to docker run.

■ Together:

ENTRYPOINT is the command, CMD is the default argument. This is the recommended pattern for application containers.

```
# Pattern 1: ENTRYPOINT + CMD (best practice for app containers) ENTRYPOINT ["java",
"-jar", "app.jar"] CMD ["--spring.profiles.active=production"] # docker run myimage ->
runs: java -jar app.jar --spring.profiles.active=production # docker run myimage
--spring.profiles.active=dev -> overrides CMD only # Pattern 2: Only CMD (flexible,
easily overridable) CMD ["java", "-jar", "app.jar"] # docker run myimage bash ->
overrides entire command with bash # Trap question: Shell form vs Exec form # Shell form
(BAD - spawns sh -c, signals not passed to app) CMD java -jar app.jar # Exec form (GOOD -
direct exec, SIGTERM reaches app for graceful shutdown) CMD ["java", "-jar", "app.jar"]
```

Tip: Always use exec form [] for ENTRYPOINT and CMD in production. Shell form prevents graceful shutdown — SIGTERM from Kubernetes does not reach the Java process.

Q: How do you reduce Docker image size beyond multi-stage builds?

■ *Concept Explanation*

■ 1 — Use Alpine / Distroless base images:

eclipse-temurin:17-jre-alpine is 80MB vs eclipse-temurin:17 at 400MB. Distroless images remove even the shell, reducing attack surface.

■ 2 — Minimize layers:

Combine RUN commands with && to create single layer instead of multiple.

■ 3 — .dockerignore:

Exclude node_modules, .git, test files, local configs from build context. Reduces context sent to Docker daemon.

■ 4 — Remove build artifacts:

Delete downloaded packages and temp files in the same RUN layer where they were created.

```
# .dockerignore .git node_modules target/ *.log *.md .env* test/ # Combined RUN layers
(reduces image layers) RUN apt-get update && \ apt-get install -y --no-install-recommends
curl && \ rm -rf /var/lib/apt/lists/* # Check image size and layers docker images
my-service docker history my-service:latest
```

7. Linux Administration (RHEL / AWS Linux)

Q: How do you check memory and CPU usage in Linux?

■ Concept Explanation

```
# CPU usage – overall and per-core top htop # Interactive, color-coded (install if not available) mpstat 1 5 # 5 samples, 1-second interval (from sysstat package) # CPU usage snapshot sar -u 1 3 # Memory usage free -h # Human-readable (MB/GB) # Columns: total / used / free / shared / buff/cache / available # 'available' is what can be used without swapping # Detailed memory breakdown cat /proc/meminfo # Per-process memory & CPU – top consumers ps aux --sort=-%mem | head -10 ps aux --sort=-%cpu | head -10 # Check if system is memory-constrained vmstat 1 5 # si/so columns = swap in/out (non-zero = memory pressure) # Check specific process pidstat -p 1 5
```

Q: What happens when disk is full? How do you handle it?

■ Root Cause Method — Identify → Diagnose → Fix → Prevent

■ Impact:

Applications fail to write logs or temp files. Docker cannot pull images or run containers. Jenkins builds fail with no space left on device. Database writes fail.

```
# Step 1: Identify which filesystem is full df -h # Look for Use% = 100% # Step 2: Find what is consuming space du -sh /* 2>/dev/null | sort -rh | head -20 # Step 3: Find large files find / -type f -size +500M 2>/dev/null | sort -k5 -rn # Step 4: Check for large log files du -sh /var/log/* # Step 5: Docker cleanup (common cause on Jenkins / EKS nodes) docker system df docker system prune -af --volumes # Step 6: Clean old journal logs journalctl --disk-usage journalctl --vacuum-size=500M # Step 7: Find deleted-but-still-open files consuming space lsof +L1 | awk '{print $7, $9}' | sort -rn | head -10
```

■ Prevent:

CloudWatch alarm on disk usage > 80%. Automated docker system prune cron job on Jenkins agents. Log rotation configured in /etc/logrotate.d/. EKS node group with adequate EBS volume size (>= 50GB).

Q: Explain systemd services.

■ Concept Explanation — Definition → How it Works → Example

■ Definition:

systemd is the init system (PID 1) on modern Linux (RHEL, Amazon Linux 2+). It manages services (units) — starting, stopping, restarting, and enabling them at boot.

■ Unit File:

A .service file in /etc/systemd/system/ defines how to run a service — what command, which user, restart policy, and dependencies.

```
# Example: /etc/systemd/system/myapp.service [Unit] Description=MyApp Java Service After=network.target [Service] User=appuser WorkingDirectory=/opt/myapp ExecStart=/usr/bin/java -jar /opt/myapp/app.jar Restart=always RestartSec=5 StandardOutput=journal StandardError=journal [Install] WantedBy=multi-user.target # Common systemd commands systemctl daemon-reload # Reload after editing unit file systemctl start myapp # Start service systemctl stop myapp # Stop service systemctl
```

```
enable myapp # Enable at boot systemctl status myapp # Check status journalctl -u myapp  
-f # Follow logs journalctl -u myapp --since today # Today's logs
```

8. Scripting — Shell & Python

Q: Write a script to check service health.

■ Concept Explanation + Hands-On

```
#!/bin/bash # service_health_check.sh – Check multiple service health endpoints
SERVICES=( "payment-service:http://10.0.10.50:8080/actuator/health"
"order-service:http://10.0.10.51:8080/actuator/health"
"auth-service:http://10.0.10.52:8080/actuator/health" ) ALERT_EMAIL="devops@company.com"
FAILED_SERVICES=() for entry in "${SERVICES[@]"; do NAME=$(echo "$entry" | cut -d':' -f1) URL=$(echo "$entry" | cut -d':' -f2-) HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" \ --max-time 5 "$URL") if [ "$HTTP_CODE" -eq 200 ]; then echo "[OK] $NAME ($HTTP_CODE)" else echo "[FAIL] $NAME ($HTTP_CODE)" FAILED_SERVICES+=("$NAME") fi done
if [ ${#FAILED_SERVICES[@]} -gt 0 ]; then echo "ALERT: Unhealthy services: ${FAILED_SERVICES[*]}" # Send alert (example using AWS SNS) aws sns publish \ --topic-arn arn:aws:sns:ap-south-1:123456789:ops-alerts \ --message "CRITICAL: ${FAILED_SERVICES[*]} health check FAILED" exit 1 fi exit 0
```

Q: How would you automate log cleanup with a script?

■ Concept Explanation

```
#!/bin/bash # log_cleanup.sh – Remove logs older than N days LOG_DIR="/var/log/myapp"
RETENTION_DAYS=7 MIN_FREE_PERCENT=20 # Check current disk usage DISK_USED=$(df "$LOG_DIR" | awk 'NR==2 {print $5}' | tr -d '%') echo "Current disk usage: ${DISK_USED}%"
# Delete logs older than retention period echo "Deleting logs older than ${RETENTION_DAYS} days from ${LOG_DIR}..." find "$LOG_DIR" -type f -name '*.log' \ -mtime +${RETENTION_DAYS} -exec rm -v {} \; # Compress logs older than 2 days but newer than retention find "$LOG_DIR" -type f -name '*.log' \ -mtime +2 -not -name '*.gz' \ -exec gzip -v {} \; # Post-cleanup disk check DISK_USED_AFTER=$(df "$LOG_DIR" | awk 'NR==2 {print $5}' | tr -d '%') echo "Disk usage after cleanup: ${DISK_USED_AFTER}%" # Crontab entry – run daily at 2 AM # 0 2 * * * /opt/scripts/log_cleanup.sh >> /var/log/cleanup.log 2>&1
```

Q: Difference between Bash and Python for automation?

■ Compare & Justify

■ Bash — Use When:

Orchestrating existing CLI tools (kubectl, aws, docker, terraform)
File operations, text processing, log parsing
System administration tasks (cron jobs, startup scripts)
Quick one-liners and pipeline chaining
Native on every Linux system — no dependencies

■ Python — Use When:

Complex data processing or business logic
Calling REST APIs with JSON parsing
Error handling, retries, and state management
Working with AWS SDK (boto3) for complex automation

Writing maintainable, testable scripts over 100 lines

■ My Rule:

If I am chaining CLI commands and file operations, Bash. If I am handling data structures, making API calls, or the script will be maintained by others, Python.

```
# Python example: Restart unhealthy ECS tasks via boto3
import boto3
ecs = boto3.client('ecs', region_name='ap-south-1')
response = ecs.list_tasks(cluster='production', desiredStatus='RUNNING')
for task_arn in response['taskArns']:
    task = ecs.describe_tasks(cluster='production', tasks=[task_arn])['tasks'][0]
    if task['healthStatus'] == 'UNHEALTHY':
        ecs.stop_task(cluster='production', task=task_arn, reason='Automated restart: unhealthy')
    print(f'Restarted: {task_arn}')
```

9. DevSecOps

Q: How does Trivy work?

■ Concept Explanation — Definition → How it Works → Example

■ Definition:

Trivy is an open-source vulnerability scanner by Aqua Security. It scans container images, filesystems, and IaC files for CVEs (Common Vulnerabilities and Exposures), misconfigurations, and exposed secrets.

■ How It Works:

Trivy downloads vulnerability databases (NVD, GitHub Advisory, etc.) and updates them locally. For a Docker image scan, it extracts the image layers, identifies OS packages (apt, yum) and language packages (Maven, pip, npm), then matches against the CVE database.

■ In My Pipeline:

Trivy scan runs before pushing to ECR. If HIGH or CRITICAL CVEs are found, the pipeline fails and the image is not deployed. This ensures no vulnerable images reach production.

```
# Scan a Docker image trivy image \ --exit-code 1 \ --severity HIGH,CRITICAL \ --format
table \ my-service:${GIT_COMMIT} # Scan and output to JSON (for integration with
dashboards) trivy image \ --format json \ --output trivy-report.json \
my-service:${GIT_COMMIT} # Scan a filesystem (IaC files, source code) trivy fs \
--exit-code 1 \ --severity HIGH,CRITICAL \ . # Scan Terraform files for misconfigurations
trivy config \ --severity HIGH,CRITICAL \ ./terraform/
```

Q: What is SAST vs DAST?

■ Compare & Justify

■ SAST — Static Application Security Testing:

Analyzes source code, bytecode, or binaries WITHOUT running the application. Finds vulnerabilities at development time (shift left). Tool in my pipeline: SonarQube. Examples: SQL injection patterns in code, hardcoded passwords, insecure crypto usage.

■ DAST — Dynamic Application Security Testing:

Tests the RUNNING application by simulating attacks from the outside. Finds vulnerabilities that only manifest at runtime. Tools: OWASP ZAP, Burp Suite. Examples: authentication bypass, XSS, runtime injection vulnerabilities.

■ My Pipeline:

SAST (SonarQube) runs at build stage before Docker build — catches code-level issues early. DAST would run in staging environment after deployment using OWASP ZAP — I have implemented this in discussion for the roadmap at Hisan Labs.

■ Complementary, Not Competing:

SAST catches 30% of vulnerabilities DAST misses (code logic). DAST catches 40% SAST misses (runtime behavior). Both are needed for comprehensive security.

Q: How do you manage secrets in production Kubernetes?

■ Top-Down Design

■ Layer 1 — Kubernetes Secrets (baseline):

Base64-encoded, stored in etcd. Accessible to pods via environment variables or volume mounts. NOT encrypted at rest by default in vanilla K8s, but EKS encrypts secrets using AWS KMS by default.

■ Layer 2 — AWS Secrets Manager (production standard):

Secrets stored encrypted in AWS Secrets Manager. External Secrets Operator syncs AWS secrets to Kubernetes Secrets automatically. Automatic rotation of DB passwords handled by AWS without any code change.

■ Layer 3 — RBAC Restriction:

Only specific service accounts can read specific secrets. No pod has access to secrets it does not need.

■ Layer 4 — Audit:

CloudTrail logs every GetSecretValue API call. Alerts fire if unexpected service account reads a secret.

```
# ExternalSecret – sync AWS Secrets Manager to K8s Secret apiVersion:
external-secrets.io/v1beta1 kind: ExternalSecret metadata: name: db-credentials
namespace: production spec: refreshInterval: 1h secretStoreRef: name:
aws-secrets-manager kind: SecretStore target: name: db-credentials-k8s data: -
secretKey: DB_PASSWORD remoteRef: key: prod/myapp/db-password property: password
```

10. Monitoring & Observability

Q: How does Prometheus collect metrics?

■ Concept Explanation — Definition → How it Works → Example

■ Pull Model:

Prometheus SCRAPES metrics from targets (pods, nodes, services). Targets expose a /metrics HTTP endpoint in Prometheus exposition format (key-value pairs with labels). Prometheus fetches this endpoint every scrape_interval (default 15 seconds).

■ Service Discovery:

In Kubernetes, Prometheus uses ServiceMonitor CRDs (Custom Resource Definitions) to automatically discover which services to scrape based on label selectors — no manual configuration needed when new services are deployed.

■ Storage:

Time-series data stored locally in TSDB (Time Series Database). Each metric has a name, labels (key-value pairs), and timestamped values. Labels enable multi-dimensional filtering.

```
# ServiceMonitor – auto-discover Spring Boot services apiVersion:
monitoring.coreos.com/v1 kind: ServiceMonitor metadata: name: payment-service-monitor
namespace: monitoring spec: selector: matchLabels: app: payment-service # Matches
services with this label endpoints: - port: http path: /actuator/prometheus interval: 15s
# Query Prometheus: HTTP error rate in last 5 minutes
rate(http_server_requests_seconds_count{ status=~"5..", job="payment-service" }[5m])
```

Q: Difference between logs, metrics, and traces?

■ Concept Explanation

■ Metrics:

Numerical time-series data. Aggregated, low storage, fast to query. Answers: Is the system healthy? What is the current request rate? CPU at 80%. Tool: Prometheus + Grafana. Best for: alerting, dashboards, capacity planning.

■ Logs:

Timestamped text records of events. High detail, high storage. Answers: What exactly happened and when? Tool: ELK Stack (Elasticsearch, Logstash, Kibana), CloudWatch Logs. Best for: debugging specific incidents.

■ Traces:

Distributed tracing tracks a single request across multiple microservices. Shows where latency is introduced across the call chain. Tool: Jaeger, AWS X-Ray, Datadog APM. Best for: identifying performance bottlenecks in distributed systems.

■ The Three Pillars:

All three are needed: metrics alert you that something is wrong, logs tell you what happened, traces tell you where in the distributed system the problem is.

Q: How did you reduce MTTR by 30%?

■ Project Walkthrough

■ Before:

Mean Time To Resolve was high because: alerts were vague (CPU high), logs were scattered across multiple pods with no aggregation, and engineers had to manually SSH into nodes to investigate.

■ Action 1 — Actionable Alerts:

Replaced generic 'CPU > 80%' alert with specific alerts: 'Payment service P99 latency > 2 seconds for 5 minutes' — gives engineers immediate context on what service and what SLO is breached.

■ Action 2 — Centralized Logging:

Deployed Fluent Bit as DaemonSet to ship all pod logs to CloudWatch Log Groups organized by service and environment. Engineers now have one place to search all service logs.

■ Action 3 — Grafana Runbook Links:

Each Grafana alert has a runbook URL in its annotation. Alert fires → Slack message includes link to runbook → engineer follows step-by-step diagnosis guide.

■ Action 4 — Datadog APM:

Added Datadog tracing to identify which downstream service was causing latency instead of manually correlating logs.

■ Result:

MTTR reduced from average 40 minutes to 28 minutes (30% reduction). Incidents that previously required 3 engineers now resolved by 1 on-call engineer using the runbook.

11. Cross / Trap Questions

Q: You said 99.9% availability — how exactly did you measure it?

■ Compare & Justify

■ Measurement Method:

Availability = (Total Time - Downtime) / Total Time × 100. I measured using Prometheus uptime metrics combined with ALB 5xx error rate as a proxy for service unavailability.

■ Definition of Downtime:

A period where error rate exceeded 5% for more than 1 minute, OR where the health check endpoint returned non-200 for more than 3 consecutive checks.

■ Tooling:

Grafana dashboard tracked monthly uptime SLO. Prometheus alert fires when error rate spikes. CloudWatch ALB metrics provided independent validation.

■ Honest Context:

99.9% uptime = 8.7 hours downtime per year, 43.8 minutes per month. In the internship project context, we achieved this over a 6-month observation period in staging and production with 3 planned maintenance windows excluded.

Watch Out: Interviewers often probe this. Be specific about how you defined and measured 'availability' — not just a number you state.

Q: Why did you choose EKS over ECS?

■ Compare & Justify

■ ECS:

AWS-native container orchestration. Simpler setup, tighter AWS integration, lower operational overhead. Best for teams that want AWS to manage everything.

■ EKS:

Managed Kubernetes. Richer ecosystem (Helm, ArgoCD, Karpenter, KEDA, any CNCF tool). Portable — same skills and manifests work on any cloud or on-prem. More control over networking, scheduling, and extensibility.

■ My Reason:

EKS was chosen because: (1) the JD and the Worldpay role require Kubernetes expertise specifically, (2) the CNCF ecosystem of tools (Prometheus, Grafana, HPA, NetworkPolicies) integrates natively, (3) Kubernetes skills are transferable and more valuable long-term than ECS-specific knowledge.

■ Trade-off Acknowledged:

EKS has higher operational complexity. For a small team running 2-3 services, ECS Fargate would be simpler. For 10+ microservices with custom autoscaling and security requirements, EKS is the right choice.

Q: You used HPA — what metrics did you specifically configure?

■ Concept Explanation

■ ApnaKart:

CPU utilization at 60% average across pods. Memory utilization at 75%. Min replicas: 2. Max replicas: 10. CPU threshold chosen based on load testing — at 60% CPU, response time started degrading.

■ Flight Reservation:

CPU at 70% for most services. Payment service: lower threshold (50%) to scale aggressively due to revenue impact of high latency. HPA tested against 5x normal load using JMeter.

■ Metrics Source:

Kubernetes Metrics Server (CPU/memory). Custom metrics via Prometheus Adapter would enable HPA on application-level metrics (requests per second, queue depth) — planned enhancement.

```
# Check HPA current state and targets kubectl get hpa -n production # Detailed HPA status
showing current vs desired metrics kubectl describe hpa payment-service-hpa -n
production # Sample output fields to understand # Current: 45% / 60% CPU (below
threshold, no scale out) # Min pods: 2 # Max pods: 10 # Replicas: 3
```

12. Rapid-Fire Round

Answer these concisely and confidently. One to three sentences maximum.

Q: What is idempotency?

A: An operation is idempotent if running it multiple times produces the same result as running it once. Terraform apply is idempotent — running it 5 times on unchanged config changes nothing. Critical for safe automation retries.

Q: What is blue-green vs canary deployment?

A: Blue-green: switch 100% of traffic at once from old to new environment — instant rollback possible. Canary: gradually shift small percentage of traffic (5%, 10%, 50%) to new version, monitoring for errors before full rollout. Canary is safer for high-risk releases.

Q: What is Infrastructure as Code?

A: Managing and provisioning infrastructure through machine-readable configuration files (Terraform, CloudFormation) instead of manual processes. Enables version control, repeatability, and automated deployment of entire environments.

Q: What is immutable infrastructure?

A: Instead of updating running servers, you replace them entirely with new ones built from the same image. Docker containers and AMIs follow this pattern. Benefits: eliminates configuration drift, makes rollback trivial, improves consistency across environments.

Q: What is a sidecar container?

A: A secondary container running alongside the main application container in the same Kubernetes pod, sharing its network and storage. Common examples: log shippers (Fluent Bit), service mesh proxies (Envoy/Istio), secret injectors (Vault Agent). I used Datadog agent as a sidecar for APM tracing.

Q: What is a Kubernetes Namespace?

A: A virtual cluster within a Kubernetes cluster used to isolate resources between teams, environments, or applications. RBAC and NetworkPolicies apply per namespace. I use namespaces: production, staging, monitoring, kube-system.

Q: What is a ConfigMap?

A: A Kubernetes object that stores non-sensitive configuration data (environment variables, config files) separately from the container image. Allows configuration changes without rebuilding images.

Q: What is the difference between a Deployment and a StatefulSet?

A: Deployment: stateless apps — pods are interchangeable, random names, any pod can be scaled/replaced freely. StatefulSet: stateful apps (databases) — pods have stable identities, ordered startup/shutdown, persistent storage per pod.

Q: What is a DaemonSet?

A: Ensures exactly one pod runs on every node in the cluster. Used for: log collectors (Fluent Bit), monitoring agents (Datadog agent, node exporter), network plugins. I used DaemonSet for Datadog and Prometheus node exporter.

Q: What is GitOps?

A: A practice where the Git repository is the single source of truth for both application code and infrastructure state. Changes to infrastructure are made via Git commits and PRs, not manual commands. Tools: ArgoCD, Flux. Benefit: audit trail, rollback via git revert, peer review for all changes.

Q: What is an Ingress Controller?

A: A Kubernetes component that implements the Ingress resource — it manages external HTTP/HTTPS access to services inside the cluster. In EKS I used the AWS Load Balancer Controller which provisions an ALB for each Ingress resource.

Q: What is kubectl top?

A: kubectl top nodes shows CPU and memory usage per node. kubectl top pods shows usage per pod. Requires Metrics Server installed in the cluster. Used for real-time resource monitoring and right-sizing resource requests/limits.

13. Smart Questions YOU Should Ask

Asking thoughtful questions shows maturity, genuine interest, and strategic thinking. Ask 2-3 of these at the end of your interview.

Q: How is your CI/CD maturity today?

Why ask: Shows you understand that CI/CD is a journey, not a one-time setup. Signals you want to contribute to improvement, not just maintain status quo. Follow-up: What is the biggest bottleneck in your current pipeline?

Q: What are the biggest reliability challenges your team faces in the boarding workflow?

Why ask: Directly references the JD's 10-minute onboarding goal. Shows you read the JD carefully and are thinking about how to contribute to that mission from day one.

Q: How is on-call structured for the S&B; Platform team?

Why ask: Shows maturity and production mindset. You are not afraid of on-call — you want to understand how responsibility is shared. Follow-up: What does a typical incident look like and how long does resolution take?

Q: What tools are you planning to adopt or evaluate in the next 6 months?

Why ask: Shows you are thinking about growth and technology direction. Reveals whether the team has a culture of continuous improvement. Good answer might mention GitOps, Karpenter, OpenTelemetry, or policy-as-code tools.

Q: How do developers and DevOps engineers collaborate here — is there a platform engineering model or is it more embedded DevOps?

Why ask: Shows awareness of different DevOps organizational models. Helps you understand your day-to-day working relationship with developers before you join.

Q: What would success look like for this role in the first 90 days?

Why ask: Classic but effective. Shows ambition and planning mindset. Gives you a clear picture of expectations so you can mentally prepare your ramp-up plan.

You've Got This, Sanket!

Worldpay | AWS DevOps Engineer | Pune

Every question is an opportunity to tell your story. You built real systems. Reduced deployment time by 77%. Achieved 98% success rates. That is not luck — that is engineering.